

Support Vector Machines (SVM)

Principios, Formulación y Aplicaciones

Prof. D.Sc. BARSEKH-ONJI Aboud

Facultad de Ingeniería
Universidad Anáhuac México

March 13, 2026

Agenda General

- 1 Motivación e Introducción
- 2 Hiperplano de Margen Máximo
- 3 Formulación Dual y Multiplicadores de Lagrange
- 4 SVM de Margen Suave (Soft Margin)
- 5 El Truco del Kernel
- 6 SVM Multi-clase y SVR
- 7 Implementación en MATLAB
- 8 Ventajas, Limitaciones y Comparativa
- 9 Resumen y Referencias

Agenda

- 1 Motivación e Introducción
- 2 Hiperplano de Margen Máximo
- 3 Formulación Dual y Multiplicadores de Lagrange
- 4 SVM de Margen Suave (Soft Margin)
- 5 El Truco del Kernel
- 6 SVM Multi-clase y SVR
- 7 Implementación en MATLAB
- 8 Ventajas, Limitaciones y Comparativa
- 9 Resumen y Referencias

¿Por qué SVM?

Contexto histórico

- Desarrollado por **Vapnik y Cortes** (1995) en los Laboratorios Bell
- Basado en la *Teoría del Aprendizaje Estadístico*
- Fundamentado en el principio de **minimización del riesgo estructural**

¿Qué lo hace especial?

- Garantías teóricas de generalización
- Eficaz en espacios de alta dimensión
- Funciona bien con muestras pequeñas
- Robusto ante *overfitting*

¿Por qué SVM?

Problema central

Dada una muestra de entrenamiento, encontrar el **hiperplano óptimo** que separe dos clases con el **máximo margen posible**.

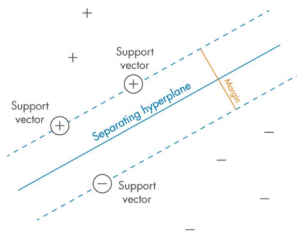


Figure 1: Hiperplano separador con margen máximo

El Problema de Clasificación Lineal

Escenario básico

Sea un conjunto de datos $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ donde:

- $\mathbf{x}_i \in \mathbb{R}^d$ — vector de características
- $y_i \in \{-1, +1\}$ — etiqueta de clase

Clasificador lineal genérico:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b) \quad (1)$$

- $\mathbf{w}$ — vector normal al hiperplano
- b — término de sesgo (*bias*)

Problema de ambigüedad

Pueden existir **infinitos hiperplanos** separadores. ¿Cuál elegir?

⇒ SVM responde: el que **maximiza el margen**.

Agenda

- 1 Motivación e Introducción
- 2 Hiperplano de Margen Máximo
- 3 Formulación Dual y Multiplicadores de Lagrange
- 4 SVM de Margen Suave (Soft Margin)
- 5 El Truco del Kernel
- 6 SVM Multi-clase y SVR
- 7 Implementación en MATLAB
- 8 Ventajas, Limitaciones y Comparativa
- 9 Resumen y Referencias

Concepto de Margen

Definición de margen

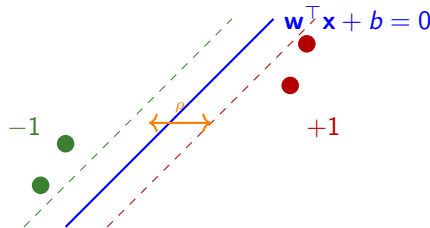
El **margen** es la distancia entre el hiperplano separador y los puntos más cercanos de cada clase.

La distancia de un punto $\mathbf{x}_i$ al hiperplano es:

$$d_i = \frac{y_i (\mathbf{w}^\top \mathbf{x}_i + b)}{\|\mathbf{w}\|} \quad (2)$$

El margen total es:

$$\rho = \frac{2}{\|\mathbf{w}\|} \quad (3)$$



Vectores de Soporte

Definición de *Support Vectors*

Los **vectores de soporte** son los puntos de entrenamiento que se encuentran exactamente sobre los hiperplanos de margen:

$$\mathbf{w}^\top \mathbf{x}_i + b = \pm 1 \quad (4)$$

Son los únicos puntos que **definen el hiperplano óptimo**.

Propiedad clave

Si se eliminan todos los puntos de entrenamiento **excepto** los vectores de soporte, el hiperplano óptimo no cambia.

Implicación práctica

SVM es **eficiente en memoria**: solo almacena los vectores de soporte, no todo el conjunto de entrenamiento.

Formulación del Problema de Optimización

Problema primal — caso separable

Maximizar el margen $\rho = 2/\|\mathbf{w}\|$ es equivalente a:

$$\min_{\mathbf{w}, b} \quad \frac{1}{2} \|\mathbf{w}\|^2 \quad (5)$$

sujeto a:

$$y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad \forall i = 1, \dots, N \quad (6)$$

- $\|\mathbf{w}\|^2 = \mathbf{w}^\top \mathbf{w}$ — norma euclidiana al cuadrado
- Las restricciones garantizan que todos los puntos estén correctamente clasificados con margen ≥ 1
- Es un problema de **programación cuadrática convexa** (QP) — solución única y global

Agenda

- 1 Motivación e Introducción
- 2 Hiperplano de Margen Máximo
- 3 Formulación Dual y Multiplicadores de Lagrange
- 4 SVM de Margen Suave (Soft Margin)
- 5 El Truco del Kernel
- 6 SVM Multi-clase y SVR
- 7 Implementación en MATLAB
- 8 Ventajas, Limitaciones y Comparativa
- 9 Resumen y Referencias

El Lagrangiano del Problema Primal

Para resolver el problema con restricciones, se introduce el **Lagrangiano**:

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^N \alpha_i \left[y_i (\mathbf{w}^\top \mathbf{x}_i + b) - 1 \right] \quad (7)$$

- $\alpha_i \geq 0$ — multiplicadores de Lagrange (uno por muestra)
- Se aplican las condiciones **KKT** (*Karush-Kuhn-Tucker*)

El Lagrangiano del Problema Primal

Condiciones KKT de optimalidad

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i \quad (8)$$

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \Rightarrow \sum_{i=1}^N \alpha_i y_i = 0 \quad (9)$$

Problema Dual

Sustituyendo las condiciones KKT en el Lagrangiano se obtiene el **problema dual**:

$$\max_{\alpha} \quad W(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \mathbf{x}_i^{\top} \mathbf{x}_j \quad (10)$$

sujeto a:

$$\alpha_i \geq 0, \quad \sum_{i=1}^N \alpha_i y_i = 0 \quad (11)$$

Problema Dual

Holgura complementaria (KKT)

$$\alpha_i \left[y_i (\mathbf{w}^\top \mathbf{x}_i + b) - 1 \right] = 0 \quad (12)$$

Si $\alpha_i > 0 \Rightarrow$ punto es vector de soporte.

Ventaja del dual

El problema dual depende solo de **productos internos** $\mathbf{x}_i^\top \mathbf{x}_j$, lo que facilita la extensión mediante **funciones Kernel**.

Clasificación con el Modelo Entrenado

Una vez resuelto el dual, se obtiene:

$$\mathbf{w}^* = \sum_{i \in SV} \alpha_i^* y_i \mathbf{x}_i \quad (13)$$

$$b^* = y_k - \mathbf{w}^{*\top} \mathbf{x}_k \quad \text{para cualquier vector de soporte } \mathbf{x}_k \quad (14)$$

La función de decisión para un nuevo punto $\mathbf{x}$ es:

$$f(\mathbf{x}) = \text{sign} \left(\sum_{i \in SV} \alpha_i^* y_i \mathbf{x}_i^\top \mathbf{x} + b^* \right) \quad (15)$$

Clasificación con el Modelo Entrenado

Observación importante

La clasificación se realiza únicamente sobre los **vectores de soporte** (donde $\alpha_i^* > 0$).
Puntos con $\alpha_i^* = 0$ no participan.

Agenda

- 1 Motivación e Introducción
- 2 Hiperplano de Margen Máximo
- 3 Formulación Dual y Multiplicadores de Lagrange
- 4 SVM de Margen Suave (Soft Margin)**
- 5 El Truco del Kernel
- 6 SVM Multi-clase y SVR
- 7 Implementación en MATLAB
- 8 Ventajas, Limitaciones y Comparativa
- 9 Resumen y Referencias

Formulación del Margen Suave

Problema primal — *Soft Margin* SVM

$$\min_{\mathbf{w}, b, \xi} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i \quad (17)$$

sujeto a:

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i \quad (18)$$

Formulación del Margen Suave

- ξ_i — variable de holgura del punto i
- $C > 0$ — parámetro de regularización
- $\xi_i = 0$: punto clasificado correctamente
- $0 < \xi_i \leq 1$: dentro del margen
- $\xi_i > 1$: clasificado incorrectamente

Rol del parámetro C

- C grande $\Rightarrow$ penaliza errores (*hard margin*)
- C pequeño $\Rightarrow$ tolera violaciones (*soft margin*)
- C se ajusta por **validación cruzada**

Problema Dual del Margen Suave

El problema dual del *Soft Margin* es:

$$\max_{\alpha} \quad \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j \mathbf{x}_i^{\top} \mathbf{x}_j \quad (19)$$

sujeto a:

$$0 \leq \alpha_i \leq C, \quad \sum_{i=1}^N \alpha_i y_i = 0 \quad (20)$$

Problema Dual del Margen Suave

Comparación con el margen duro

Formulación	Restricción en α_j	Tolerancia a ruido
Margen Duro	$\alpha_j \geq 0$	No
Margen Suave	$0 \leq \alpha_j \leq C$	Sí

La única diferencia es la cota superior C en los multiplicadores.

Agenda

- 1 Motivación e Introducción
- 2 Hiperplano de Margen Máximo
- 3 Formulación Dual y Multiplicadores de Lagrange
- 4 SVM de Margen Suave (Soft Margin)
- 5 El Truco del Kernel**
- 6 SVM Multi-clase y SVR
- 7 Implementación en MATLAB
- 8 Ventajas, Limitaciones y Comparativa
- 9 Resumen y Referencias

Limitaciones del Clasificador Lineal

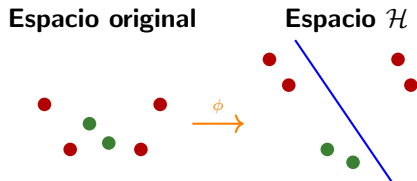
Problema con datos no lineales

Muchos problemas reales **no son linealmente separables** en el espacio original $\mathbb{R}^d$.

Idea: mapear los datos a un espacio de mayor dimensión $\mathcal{H}$ donde sí sean separables:

$$\phi : \mathbb{R}^d \rightarrow \mathcal{H} \quad (21)$$

Se aplica SVM lineal en $\mathcal{H}$.



El Truco del Kernel

Observación fundamental

El problema dual SVM solo depende de **productos internos**:

$$\mathbf{x}_i^\top \mathbf{x}_j \implies \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j) \quad (22)$$

Función Kernel

Se define la función Kernel como:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j) \quad (23)$$

El **truco del kernel** permite calcular el producto interno en $\mathcal{H}$ **sin calcular explícitamente** $\phi(\mathbf{x})$.



Kernels Más Utilizados

La función de decisión con kernel queda:

$$f(\mathbf{x}) = \text{sign} \left(\sum_{i \in SV} \alpha_i^* y_i K(\mathbf{x}_i, \mathbf{x}) + b^* \right) \quad (24)$$

Kernel	Fórmula $K(\mathbf{x}_i, \mathbf{x}_j)$	Parámetros
Lineal	$\mathbf{x}_i^\top \mathbf{x}_j$	—
Polinomial	$(\mathbf{x}_i^\top \mathbf{x}_j + c)^d$	$c \geq 0, d \in \mathbb{Z}^+$
RBF (Gaussiano)	$\exp(-\gamma \ \mathbf{x}_i - \mathbf{x}_j\ ^2)$	$\gamma > 0$
Sigmoide	$\tanh(\kappa \mathbf{x}_i^\top \mathbf{x}_j + \theta)$	κ, θ

Kernels Más Utilizados

Condición de Mercer

Una función K es un kernel válido si y solo si la matriz de Gram $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$ es **semidefinida positiva**.

Kernel RBF: Intuición Geométrica

El kernel **RBF** (*Radial Basis Function*) o Gaussiano es el más empleado en la práctica:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2), \quad \gamma > 0 \quad (25)$$

Interpretación

Mide la **similitud** entre dos puntos: decrece exponencialmente con la distancia al cuadrado entre ellos.

- γ pequeño $\Rightarrow$ vecindad amplia (*underfitting*)
- γ grande $\Rightarrow$ vecindad estrecha (*overfitting*)

Selección de hiperparámetros

Para SVM con RBF se ajustan **dos parámetros**:

- C — regularización del margen
- γ — radio del kernel

Se recomienda búsqueda en grilla logarítmica con **validación cruzada** k -fold.

Agenda

- 1 Motivación e Introducción
- 2 Hiperplano de Margen Máximo
- 3 Formulación Dual y Multiplicadores de Lagrange
- 4 SVM de Margen Suave (Soft Margin)
- 5 El Truco del Kernel
- 6 SVM Multi-clase y SVR**
- 7 Implementación en MATLAB
- 8 Ventajas, Limitaciones y Comparativa
- 9 Resumen y Referencias

Extensión a Multi-clase

SVM binario $\rightarrow$ multi-clase

SVM es inherentemente **binario**. Se extiende a K clases mediante estrategias combinatorias.

Uno contra el resto (OvR):

- K clasificadores SVM
- El clasificador k separa la clase k de todas las demás
- Se asigna la clase con mayor valor de $f_k(\mathbf{x})$

Complejidad: K problemas QP

Uno contra uno (OvO):

- $\binom{K}{2}$ clasificadores SVM
- Cada clasificador separa un par de clases
- Se asigna la clase por **votación mayoritaria**

Complejidad: $K(K - 1)/2$ problemas QP

Extensión a Multi-clase

Recomendación práctica

OvO es más común en implementaciones (e.g., MATLAB, scikit-learn) por su mejor estabilidad numérica con clases desbalanceadas.

Support Vector Regression (SVR)

SVM puede adaptarse para **regresión** con la formulación ϵ -SVR:

$$\min_{\mathbf{w}, b, \xi} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N (\xi_i + \xi_i^*) \quad (26)$$

sujeto a:

$$|y_i - (\mathbf{w}^\top \mathbf{x}_i + b)| \leq \epsilon + \xi_i^{(*)} \quad (27)$$

Tubo ϵ -insensible

Las predicciones dentro del tubo de ancho 2ϵ **no generan error**. Solo los puntos fuera del tubo contribuyen a la función objetivo.

- ϵ — anchura del tubo de tolerancia
- C — regularización (igual que en clasificación)
- ξ_i, ξ_i^* — holguras superior e inferior

Agenda

- 1 Motivación e Introducción
- 2 Hiperplano de Margen Máximo
- 3 Formulación Dual y Multiplicadores de Lagrange
- 4 SVM de Margen Suave (Soft Margin)
- 5 El Truco del Kernel
- 6 SVM Multi-clase y SVR
- 7 Implementación en MATLAB**
- 8 Ventajas, Limitaciones y Comparativa
- 9 Resumen y Referencias

SVM Binario en MATLAB — Entrenamiento

```
1 % Cargar o generar datos
2 load fisheriris;
3 X = meas(51:end, 1:2);           % 2 características, 2 clases
4 Y = categorical(species(51:end));
5
6 % Dividir en entrenamiento/prueba (70/30)
7 cv = cvpartition(Y, 'HoldOut', 0.3);
8 X_train = X(cv.training, :);
9 Y_train = Y(cv.training);
10 X_test  = X(cv.test, :);
11 Y_test  = Y(cv.test);
12
13 % Entrenar SVM con kernel RBF
14 svmModel = fitcsvm(X_train, Y_train, ...
15     'KernelFunction', 'rbf', ...
16     'BoxConstraint', 1, ...      % parametro C
17     'KernelScale', 'auto', ...  % gamma automatico
18     'Standardize', true);
```



SVM Binario en MATLAB — Evaluación

```
1 % Prediccion en conjunto de prueba
2 Y_pred = predict(svmModel, X_test);
3
4 % Matriz de confusion
5 confMat = confusionmat(Y_test, Y_pred);
6 disp('Matriz de confusion:'); disp(confMat);
7
8 % Metricas de desempeno
9 accuracy = sum(Y_pred == Y_test) / numel(Y_test) * 100;
10 fprintf('Exactitud: %.2f%%\n', accuracy);
11
12 % Visualizar frontera de decision
13 figure;
14 h = 0.02;
15 [x1g, x2g] = meshgrid(...
16     min(X(:,1))-1:h:max(X(:,1))+1, ...
17     min(X(:,2))-1:h:max(X(:,2))+1);
18 Xgrid = [x1g(:), x2g(:)];
```



Selección de Hiperparámetros con Búsqueda en Grilla

```
1 % Grilla logaritmica de C y gamma
2 C_grid      = logspace(-2, 3, 12);    % 0.01 a 1000
3 gamma_grid  = logspace(-4, 1, 12);    % 0.0001 a 10
4
5 best_acc = 0;
6 best_C = 1; best_g = 1;
7
8 for C = C_grid
9     for g = gamma_grid
10         mdl = fitcsvm(X_train, Y_train, ...
11             'KernelFunction', 'rbf', ...
12             'BoxConstraint', C, ...
13             'KernelScale', 1/sqrt(2*g), ...
14             'Standardize', true, ...
15             'CrossVal', 'on', ...
16             'KFold', 5);
17         acc = 1 - kfoldLoss(mdl);
18         if acc > best_acc
```



SVR en MATLAB

```
1 % Datos de regresion: prediccion de precio
2 % X = características, y = variable objetivo continua
3 load carsmall
4 X_reg = [Horsepower, Weight];
5 y_reg = MPG;
6
7 % Eliminar filas con NaN
8 idx = ~any(isnan([X_reg, y_reg]), 2);
9 X_reg = X_reg(idx,:); y_reg = y_reg(idx);
10
11 % Entrenar epsilon-SVR
12 svrModel = fitrsvm(X_reg, y_reg, ...
13     'KernelFunction', 'rbf', ...
14     'BoxConstraint', 10, ... % C
15     'Epsilon', 0.1, ... % epsilon del tubo
16     'KernelScale', 'auto', ...
17     'Standardize', true);
18
```



Agenda

- 1 Motivación e Introducción
- 2 Hiperplano de Margen Máximo
- 3 Formulación Dual y Multiplicadores de Lagrange
- 4 SVM de Margen Suave (Soft Margin)
- 5 El Truco del Kernel
- 6 SVM Multi-clase y SVR
- 7 Implementación en MATLAB
- 8 Ventajas, Limitaciones y Comparativa**
- 9 Resumen y Referencias

Ventajas y Limitaciones de SVM

Ventajas:

- Efectivo en **alta dimensionalidad**
- Eficiente en memoria (solo vectores de soporte)
- Garantías teóricas sólidas (*VC dimension*)
- Flexible mediante elección de kernel
- Robusto ante *overfitting* (margen máximo)
- Único mínimo global (problema convexo)

Limitaciones:

- Costoso en entrenamiento: $\mathcal{O}(N^2)$ a $\mathcal{O}(N^3)$
- No escala bien a millones de muestras
- Selección de kernel y C , γ es crítica
- No produce probabilidades directamente
- Sensible a la escala de las características
- Difícil interpretación (caja negra con kernel)

Ventajas y Limitaciones de SVM

Regla de escala

Siempre normalizar las características antes de entrenar un SVM. La distancia euclidiana (y el kernel RBF) es sensible a la magnitud de las variables.

SVM vs. Otras Técnicas de Clasificación

Método	Lineal	Kernel	Escala	Probabilidades	Interpretable
SVM (lineal)	Sí	No	Media	No	Medio
SVM (RBF)	No	Sí	Media	No	Bajo
Regresión Log.	Sí	No	Alta	Sí	Alto
Árbol de dec.	No	No	Alta	Sí	Alto
Random Forest	No	No	Alta	Sí	Bajo
Red Neuronal	No	No	Media	Sí	Muy bajo

SVM vs. Otras Técnicas de Clasificación

Cuando SVM es preferible

- Datos con alta relación dimensiones/muestras ($d \gg N$)
- Problemas con estructura clara de separación pero no lineal
- Cuando se requieren garantías teóricas de generalización

Agenda

- 1 Motivación e Introducción
- 2 Hiperplano de Margen Máximo
- 3 Formulación Dual y Multiplicadores de Lagrange
- 4 SVM de Margen Suave (Soft Margin)
- 5 El Truco del Kernel
- 6 SVM Multi-clase y SVR
- 7 Implementación en MATLAB
- 8 Ventajas, Limitaciones y Comparativa
- 9 Resumen y Referencias**

Resumen del Módulo

Conceptos clave cubiertos:

- 1 Hiperplano de margen máximo
- 2 Vectores de soporte
- 3 Formulación primal y dual (Lagrangiano + KKT)
- 4 *Soft Margin* y parámetro C
- 5 Truco del Kernel y condición de Mercer
- 6 Kernels: lineal, polinomial, RBF, sigmoide
- 7 SVM multi-clase: OvR y OvO
- 8 *Support Vector Regression* (SVR)
- 9 Implementación en MATLAB

Resumen del Módulo

Fórmula central — Función de decisión

$$f(\mathbf{x}) = \text{sign} \left(\sum_{i \in SV} \alpha_i^* y_i K(\mathbf{x}_i, \mathbf{x}) + b^* \right)$$

Flujo de trabajo SVM

- 1 Normalizar datos
- 2 Seleccionar kernel
- 3 Ajustar C y γ (CV k -fold)
- 4 Entrenar y evaluar



Referencias

Textos fundamentales

- **Cortes & Vapnik (1995)**. Support-Vector Networks. *Machine Learning*, 20(3), 273–297.
- **Vapnik, V. (1998)**. *Statistical Learning Theory*. Wiley-Interscience.
- **Schölkopf & Smola (2002)**. *Learning with Kernels*. MIT Press.
- **Bishop, C.M. (2006)**. *Pattern Recognition and Machine Learning*. Springer. Cap. 7.

Referencias

Documentación MATLAB

- <https://www.mathworks.com/help/stats/fitcsvm.html>
- <https://www.mathworks.com/help/stats/fitrsvm.html>